

Encontrar Regiones de Operación Robustas en Datos de 5–10 Dimensiones

Manual de Referencia Avanzado en Python para Ingeniería de Robustez y Superficies de Respuesta
· Edición revisada

Resumen rápido (TL;DR): Primero construye un modelo sustituto (*surrogate*) rápido, luego explóralo. Con 5–10 variables continuas, ajusta un Proceso Gaussiano (`GaussianProcessRegressor`) o un Random Forest sobre tus datos medidos, y sondea el sustituto con perturbaciones de Monte Carlo para puntuar cada punto de operación candidato mediante el objetivo robusto **media $\pm k \cdot \sigma$** (Beyer & Sendhoff, 2007). Robusto = baja varianza de salida predicha bajo pequeñas perturbaciones de entrada, no solo respuesta media óptima. Usa **HiPlot + coordenadas paralelas de Plotly** como herramienta visual principal, con pairplots de Seaborn y contornos tipo "ventana de operación" estilo Quality-by-Design (QbD). Evita los gráficos de radar y t-SNE ingenuos para este problema.

1. Fundamentos Clave del Análisis de Robustez

La pregunta de robustez es fundamentalmente sobre la **varianza local de la salida**, no sobre el óptimo global aislado. Una combinación "robusta" de entradas es aquella donde pequeñas perturbaciones ϵ alrededor de un punto candidato x^* dejan la salida $y(x^* + \epsilon)$ prácticamente sin cambios.

Geométricamente, buscamos una **meseta** (región de bajo gradiente) de la superficie de respuesta, en lugar de un pico estrecho que se degrada drásticamente ante cualquier desviación.

Esto reformula la tarea en tres flujos de trabajo paralelos:

1. **Estimación del sistema:** construir una aproximación continua $\hat{y}(x)$ a partir de datos reales mediante un modelo sustituto.
2. **Cálculo de la sensibilidad local:** evaluar y propagar la varianza alrededor de cada candidato de operación.
3. **Visualización de alta dimensión:** representar e identificar espacialmente dónde se ubican las regiones de baja sensibilidad dentro del espacio de entrada.

2. Visualización Multidimensional (5–10 Variables)

Coordenadas Paralelas (Herramienta de Cabecera)

Cada fila de tu conjunto de datos se convierte en una polilínea que cruza un eje vertical por cada variable. Al filtrar o interactuar con el eje de salida (técnica conocida como *brushing*), puedes descubrir los rangos permitidos de entrada.

```
import plotly.express as px

fig = px.parallel_coordinates(
    df,
    color="output",
    color_continuous_scale=px.colors.diverging.Tealrose,
```

```
color_continuous_midpoint=df["output"].median(),
)
fig.show()
```

Uso de HiPlot (Meta AI Research): ideal para manejar hasta medio millón de filas directamente en Jupyter, permitiendo reordenar ejes dinámicamente con un clic y limpiar valores atípicos.

```
import hiplot as hip

hip.Experiment.from_dataframe(df).display()
```

Cómo leerlas para robustez: pinta el eje de salida hasta una banda de tolerancia estrecha. Las líneas seleccionadas revelarán los rangos de entrada requeridos. Si al acotar la salida una variable de entrada x_j mantiene un rango amplio de dispersión, el sistema es robusto frente a variaciones en x_j . Si por el contrario la entrada se contrae obligatoriamente a un rango milimétrico, x_j es un factor crítico y altamente sensible.

Matriz de Dispersión (Pair Plot)

El uso de `seaborn.pairplot(df, hue="output_bin", diag_kind="kde", corner=True)` genera una cuadrícula compacta de gráficos de dispersión bidimensionales. Aunque una matriz de 10×10 variables es densa, resulta imbatible para detectar interacciones por pares, restricciones físicas acopladas y distribuciones marginales en la diagonal principal.

ADVERTENCIA CRÍTICA · EVITA RADAR Y T-SNE

Gráficos de radar: el área del polígono se distorsiona de forma cuadrática y depende del orden arbitrario de los ejes. No los uses para exploración cuantitativa.

t-SNE / UMAP: preservan la estructura vecinal local de forma estocástica, pero destruyen las distancias globales. La proximidad en un mapa t-SNE no equivale a proximidad en el espacio físico de ingeniería, lo cual invalida cualquier análisis de perturbación ϵ .

MATIZ RESTAURADO · EL PAPEL REAL DE PCA

PCA es preferible a t-SNE/UMAP porque su mapeo lineal e invertible *no miente sobre las distancias*. Pero PCA **no es** el método de robustez — es solo una visualización/preprocesado honesto. El análisis de robustez lo hace el bucle Monte Carlo sobre el sustituto (Sección 3). PCA es opcional; no lo trates como la solución.

Gráficos de Contorno y Cortes 2D (Condicionamiento)

Esta técnica consiste en congelar las variables menos influyentes en sus niveles nominales (o medianas) y evaluar el modelo sustituto sobre una malla fina de las dos variables más críticas. Es el pilar fundamental del concepto *Design Space* de la industria farmacéutica y de manufactura avanzada.

3. Metodología de Optimización y Diseño Robusto

Análisis de Sensibilidad Global con SALib

Para combatir la maldición de la dimensionalidad se usa SALib para cuantificar qué variables dominan el sistema. La API moderna `ProblemSpec` estructura el análisis:

```
import numpy as np
from SALib import ProblemSpec
from sklearn.gaussian_process import GaussianProcessRegressor

# Ajustar sustituto sobre los datos medidos reales
gpr = GaussianProcessRegressor().fit(X_meas, y_meas)

sp = ProblemSpec({
    "names": ["x1", "x2", "x3", "x4", "x5", "x6"],
    "bounds": [[lo, hi] for lo, hi in zip(X_meas.min(0), X_meas.max(0))],
    "outputs": ["Y"],
})

(sp.sample_sobol(1024)
 .evaluate(lambda X: gpr.predict(X))
 .analyze_sobol())

total_Si, first_Si, second_Si = sp.to_df()
```

Interpretación: el índice de primer orden (S_1) es la varianza explicada por la variable por sí sola. El índice de orden total (S_T) incluye todas sus interacciones con las demás. Si $S_T \gg S_1$, la variable participa en acoplamientos fuertes y no puede ajustarse de forma aislada.

ADVERTENCIA RESTAURADA · DATOS PASIVOS ≠ EXPERIMENTO DISEÑADO

Esta es la trampa nº1 con mediciones reales. Si tus variables están correlacionadas en los datos históricos (p. ej., siempre subiste temperatura y presión juntas), entonces: (1) el sustituto **no es fiable fuera de la nube de datos observada**, y (2) los índices de Sobol asumen **entradas independientes** y pueden volverse engañosos. Mitigación: restringe el análisis a los límites con densidad de datos adecuada, o usa *Shapley effects* (manejan correlación) en vez de Sobol clásico.

ADVERTENCIA RESTAURADA · DISCIPLINA DE TAMAÑO DE MUESTRA

El estimador Sobol de Saltelli necesita $N \cdot (2k+2)$ evaluaciones del modelo (≈ 12.000 para $k=10$, $N=1024$). Si tu dataset tiene **menos de ~50 filas**, omite Sobol — no se cumplen sus supuestos de muestreo — y quédate solo con Morris, apoyándote en el RMSE leave-one-out del GP como medida de calidad. Si puedes, recolecta puntos con un diseño Box-Behnken o LHS para llenar vacíos.

Puntuación de Robustez mediante Perturbación Monte Carlo

Evaluar un candidato x^* no se limita a calcular $\hat{y}(x^*)$. Se debe simular el ruido real del proceso inyectando perturbaciones estocásticas controladas. **Código completo (sin cortes):**

```
import numpy as np

def robustness_score(x_star, surrogate_model, sigma_eps, n=500, k=2):
    # Generar perturbaciones normales simulando la tolerancia física real
    eps = np.random.normal(0, sigma_eps, size=(n, x_star.size))
    Y_perturbed = surrogate_model.predict(x_star + eps)

    media = Y_perturbed.mean()
    desviacion = Y_perturbed.std()

    # Objetivo robusto canónico (Beyer & Sendhoff, 2007)
    objetivo_robusto = media + k * desviacion

    return media, desviacion, objetivo_robusto
```

Usa el **ruido/tolerancia físico real** como `sigma_eps`, no un valor arbitrario. Tu región robusta es el subconjunto donde la media cumple el objetivo y la desviación $< \tau$ (o equivalentemente $P(\text{en-especificación}) > 1-\alpha$).

DIFERENCIA TEÓRICA VITAL · EPISTÉMICA VS. ALEATORIA

`gpr.predict(X, return_std=True)` devuelve la incertidumbre del modelo (**epistémica**) — qué tan seguro está el algoritmo según la densidad de datos cercanos. **No** representa la varianza del sistema ante ruido de entrada (**aleatoria**). Para medir robustez operativa es **estrictamente obligatorio** el bucle de Monte Carlo de arriba.

MATIZ RESTAURADO · "INTERPOLACIÓN EXACTA" DEL GP

Un Proceso Gaussiano solo interpola *exactamente* si no incluye término de ruido. Con datos físicos medidos casi siempre quieres añadir un `WhiteKernel` (o un `alpha > 0`), y entonces el GP **suaviza en vez de interpolar** — que es lo correcto con ruido de medición. No esperes que la superficie pase por todos tus puntos.

DECISIÓN RESTAURADA · EL UMBRAL τ ES TUYO

La matemática te entrega $\hat{\sigma}(x)$ por candidato, pero **cuán pequeño debe ser $\hat{\sigma}$ para llamar "robusta" a una región es una decisión de negocio/ingeniería**, no un valor que salga del modelo. Hazlo explícito y documentalo.

4. Tabla Resumen de Herramientas Recomendadas

Tarea específica	Librería recomendada	Propósito operativo
Exploración visual interactiva	HiPlot / Plotly Express	Brushing multidimensional y filtrado rápido de rangos estables.
Distribuciones por pares e interacciones	seaborn (pairplot)	Correlaciones directas y fronteras físicas en 2D.
Cribado inicial de variables	SALib (Método de Morris)	

Tarea específica	Librería recomendada	Propósito operativo
		Descarte rápido de variables irrelevantes a bajo costo (N=20–50).
Análisis de sensibilidad global	SALib (Índices de Sobol)	Descomposición de varianza e identificación de acoplamientos (S_T).
Modelado sustituto suave / probabilístico	scikit-learn (GaussianProcess)	Superficies continuas + métricas de error. Añade WhiteKernel si hay ruido.
Modelado sustituto no lineal / discontinuo	lightgbm / xgboost	Sustitutos rápidos para comportamientos rugosos o discontinuos.

5. Flujo de Trabajo Secuencial Sugerido

1. **Fase estática inicial:** ejecuta un pairplot de Seaborn y una matriz de correlación de Spearman para limpiar datos imposibles o duplicados.
2. **Ajuste de sustituto base:** ajusta un `RandomForestRegressor` rápido y revisa las importancias nativas como validación preliminar.
3. **Cribado formal (Morris):** usa muestreo de Morris en SALib para identificar variables de efecto despreciable y congélalas en sus medianas. Reduce de ~10 variables a las 4–5 dominantes.
4. **Modelado de alta fidelidad:** ajusta un Proceso Gaussiano con kernel Matern 5/2 (+ WhiteKernel si hay ruido) sobre las variables activas. Valida con residuos y un gráfico de paridad.
5. **Simulación de perturbación:** ejecuta el Monte Carlo inyectando las desviaciones de tolerancia reales sobre una malla fina del espacio reducido.
6. **Maapeo del Design Space (QbD):** genera contornos 2D de $P(\text{en-especificación}) > 0.95$ usando las dos variables de mayor S_T, con pequeños múltiplos para niveles discretos de una tercera variable.
7. **Confirmación física:** valida con 3–5 corridas reales dentro de la región robusta predicha (y 1–2 fuera) antes de fijar la operación.